

UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

Facultad de Ingeniería

Ingeniería en Desarrollo de Software y Ciencia de Datos

Programación II

**SISTEMA PARA ADMINISTRACIÓN
DE ASOCIACIÓN COMUNAL**

Segundo avance del proyecto

Fotografía	Nombre completo	CIF	¿Participó?
	Carlos Josué Samayoa Contreras	2025010313	SÍ
	Josué Jeremías Castillo Nieves	2026010141	SÍ
	Jonathan Alexis Amaya Hernández	2026020124	SÍ
<i>Sin fotografía</i>	Génesis Michelle Flores Cruz	2026011327	NO

	David Rafael Coto Garza	2026010490	sí
	Rodrigo Gilberto Sánchez Ávalos	2026011711	NO
	Gerson Otoniel Bermúdez Ramírez	2026020119	NO
<i>Sin fotografía</i>	Edgard Antonio Mendoza Vaquero	2026011014	NO

Facilitador: Ing. Daniel Enrique Guevara Gómez

San Salvador, El Salvador · 2026

Fecha de entrega: 27 de septiembre de 2026, 13:00

Contenido

1. Portada y participación del equipo	5
2. Objetivo general	6
3. Objetivos específicos del segundo avance	6
4. Distribución del equipo Scrum	7
5. Roles y funciones del sistema	9
6. Requerimientos del sistema e historias de usuario	10
6.1 Requerimientos funcionales consolidados	11
6.2 Requerimientos no funcionales	12
7. Alcance, limitaciones y límites	13
7.1 Funcionalidad implementada y verificada	13
7.2 Funcionalidad parcialmente implementada	13
7.3 Funcionalidad pendiente para avances posteriores	13
7.4 Fuera del alcance del proyecto	14
8. Planificación	14
8.1 Primer avance — análisis y diseño	14
8.2 Segundo avance — implementación y evidencia técnica	14
8.3 Ciclo posterior a la integración y tercer avance — planificado.....	15
9. Cronograma — diagrama de Gantt	17
10. Entradas y salidas del sistema	19
11. Declaración de entidades del sistema.....	20
12. El proyecto: evolución técnica del segundo avance	21
12.1 Tecnología empleada	21
12.2 Aclaraciones técnicas validadas por el facilitador.....	21
12.3 Evolución del primer avance y diseño de clases	22
12.4 Encapsulamiento	22
12.5 Herencia	23
12.6 Polimorfismo	24
12.7 Programación concurrente	25
12.8 Clases abstractas	26
12.9 Colecciones.....	27

12.10 Arquitectura por capas.....	28
12.11 Persistencia mediante archivos .dat	29
12.12 Versionamiento con Git	30
12.13 Integración y funcionamiento del proyecto.....	30
12.14 Documentación técnica y preparación de la presentación.....	31
13. Conclusiones.....	32
14. Bibliografía	33

1. Portada y participación del equipo

La portada de este documento presenta a los ocho integrantes del grupo con su fotografía, su nombre completo, su CIF y la declaración de participación correspondiente a este segundo avance. Participaron cuatro de los ocho integrantes.

La declaración de participación se corresponde con la evidencia que el propio repositorio y el tablero de Jira exponen, de modo que puede verificarse de forma independiente:

Integrante	Participación	Evidencia verificable y responsabilidad asignada
Carlos Josué Samayoa Contreras	Sí	Autor de los doce commits de la rama del avance y de los cinco documentos de evidencia técnica. Reportero de la épica SCRUM-310 y de sus tickets. Responsable de la arquitectura y de las decisiones de diseño del avance, de la reconciliación del backlog histórico contra el código, y de la coordinación con el revisor técnico para la integración. Responsable designado del desarrollo, de la evolución del código para cumplir los criterios de la rúbrica y de la corrección de defectos.
Josué Jeremías Castillo Nieves	Sí	Noventa commits en el repositorio bajo la cuenta MiasCN; revisor técnico de los diez tickets del avance.
Jonathan Alexis Amaya Hernández	Sí	Reporte y seguimiento del defecto SCRUM-159 sobre el dimensionamiento de la ventana principal. Responsable designado de QA funcional, validación posterior a la integración y despliegue de avances y correcciones relacionados con esta entrega.
David Rafael Coto Garza	Sí	QA · Aplicación móvil: responsable de la revisión funcional de la interfaz y comportamiento del cliente móvil, identificación de inconsistencias de uso o presentación y apoyo en su explicación y demostración durante el avance.

Tabla 1. Participación declarada, evidencia verificable y responsabilidad asignada.

La tabla distingue dos planos. La evidencia verificable corresponde a trabajo ya ejecutado y comprobable en el repositorio o en el tablero; la responsabilidad designada corresponde a la asignación formal de tareas para esta entrega, algunas de las cuales se ejecutan con posterioridad a la integración del avance.

Conviene precisar una distinción que el historial de Git puede hacer ambigua. El repositorio conserva dos commits anteriores a este avance realizados por la cuenta G3rze, correspondiente a Gerson Otoniel Bermúdez Ramírez. Se trata de contribuciones históricas del primer avance y no modifican la declaración de participación de este segundo avance, que se refiere exclusivamente al trabajo comprendido entre el 12 y el 14 de septiembre de 2026.

2. Objetivo general

Desarrollar una aplicación de escritorio en Java, respaldada por una base de datos MySQL, que centralice y controle la información administrativa de una asociación comunal, permitiendo gestionar el censo de miembros, los cargos de la directiva, las aportaciones mensuales, los proyectos de mejora, las reuniones y los procesos de votación; con mecanismos de validación, trazabilidad y control de acceso que contribuyan a una administración ordenada, transparente y verificable.

El objetivo general se conserva sin variaciones respecto del primer avance, por tratarse del propósito que orienta la totalidad del proyecto. Lo que evoluciona de un avance a otro son los objetivos específicos y el grado de implementación alcanzado.

3. Objetivos específicos del segundo avance

El primer avance se concentró en el análisis y el diseño. Este segundo avance tiene como propósito llevar ese diseño a una implementación que demuestre, sobre el código real del sistema, los conceptos de programación orientada a objetos que la asignatura evalúa. En consecuencia, se plantean los objetivos siguientes:

- Evolucionar el diseño de clases planteado en el primer avance, incorporando una jerarquía de dominio que represente con mayor fidelidad a las personas vinculadas a la asociación.
- Demostrar el encapsulamiento como mecanismo de protección de las reglas del negocio, y no únicamente como una convención de escritura de atributos privados.
- Implementar una relación de herencia que responda a una necesidad real del dominio, distinguiendo a quien es asociado de quien únicamente habita una vivienda.
- Ejercer el polimorfismo de modo que la capa de servicios opere sobre un contrato de acceso a datos sin conocer la tecnología de persistencia que lo respalda.
- Incorporar una clase abstracta que concentre el comportamiento común de la persistencia en archivos y delegue en sus subclases únicamente lo que varía.
- Implementar la persistencia mediante archivos .dat como origen de datos autónomo, en coexistencia con la base de datos MySQL ya existente.
- Justificar técnicamente la concurrencia presente en el sistema, explicando el problema concreto de la asociación que cada mecanismo resuelve.
- Justificar la elección de las estructuras de datos empleadas en función de la información que el sistema administra.
- Documentar la arquitectura por capas del sistema y verificar que las reglas de dependencia entre ellas se cumplen en el código.
- Validar la integración y el funcionamiento del proyecto mediante una compilación reproducible y una batería de pruebas automatizadas.

4. Distribución del equipo Scrum

El equipo mantiene la distribución de roles definida en el primer avance. La coordinación del equipo, en calidad de Scrum Master, corresponde a Josué Jeremías Castillo Nieves.

N.º	Nombre completo	Rol Scrum	Rol técnico
1	Carlos Josué Samayoa Contreras	Product Owner	Analista
2	Josué Jeremías Castillo Nieves	Scrum Master (coordinador)	Desarrollador
3	Josué Jeremías Castillo Nieves	Developer	Desarrollador frontend
4	Gerson Otoniel Bermúdez Ramírez	Developer	Desarrollador backend
5	Rodrigo Gilberto Sánchez Ávalos	Developer	Desarrollador full stack
6	Edgard Antonio Mendoza Vaquero	Developer	Base de datos
7	David Rafael Coto Garza	QA	QA · Aplicación móvil
8	Jonathan Alexis Amaya Hernández	QA	QA · Validación funcional y soporte de despliegue
9	Génesis Michelle Flores Cruz	Business Analyst	Analista de negocio
10	Carlos Josué Samayoa Contreras	Architect	Arquitecto

Tabla 2. Distribución de roles Scrum y roles técnicos del equipo.

El documento distingue deliberadamente tres planos que no deben confundirse: el rol Scrum asignado, el rol técnico correspondiente y la participación efectivamente registrada en este avance. Un integrante puede tener un rol asignado desde la planificación inicial sin haber registrado actividad en el período evaluado, y el documento refleja ambas cosas por separado para que la información sea verificable.

Para esta entrega el equipo precisó el reparto de responsabilidades que rige el ciclo de trabajo, desde la definición del alcance hasta la publicación de las correcciones.

Responsable	Ámbito asignado para esta entrega
Carlos Josué Samayoa Contreras	Arquitectura y decisiones de diseño. Desarrollo e implementación del código. Evolución del sistema para cumplir los criterios de la rúbrica del avance. Gestión y normalización del backlog, incluida la reconciliación de los requisitos históricos contra el código real. Elaboración de la documentación técnica y construcción del documento de entrega, mediante la recopilación y verificación de la evidencia procedente del historial del repositorio, del tablero de Jira y de los documentos del avance anterior. Coordinación de los cambios de código con el revisor para su integración. Corrección de los defectos reportados.
Josué Jeremías Castillo Nieves	Coordinación del equipo como Scrum Master. Revisión técnica de los cambios antes de su integración a la rama principal de desarrollo. Aprobación de la incorporación.
Jonathan Alexis Amaya Hernández	Aseguramiento de la calidad funcional de la aplicación. Revisión del build integrado. Validación de los avances. Despliegue de los avances y de las correcciones. Revalidación de lo corregido. Registro de los defectos encontrados.
David Rafael Coto Garza	QA · Aplicación móvil: responsable de la revisión funcional de la interfaz y comportamiento del cliente móvil, identificación de inconsistencias de uso o presentación y apoyo en su explicación y demostración durante el avance.

Tabla 3. Ámbito de responsabilidad asignado para esta entrega.

El reparto obedece a una separación deliberada: quien desarrolla no valida su propio trabajo ni lo publica. Carlos Josué Samayoa Contreras construye y corrige; Josué Jeremías Castillo Nieves revisa e integra; Jonathan Alexis Amaya Hernández despliega y valida. Jonathan no asume el desarrollo de las correcciones, sino su verificación y publicación. David Rafael Coto Garza asume la revisión funcional de la aplicación móvil, aportando una validación independiente de su interfaz y comportamiento.

5. Roles y funciones del sistema

Los roles siguientes corresponden a los perfiles de acceso con que operan los usuarios de la aplicación. Se encuentran definidos en el código, en la constante ROLES_BASE de la clase RolService, y protegidos frente a modificación o eliminación por tratarse de perfiles estructurales del sistema.

Rol del sistema	Descripción	Funciones autorizadas
Administrador	Responsable técnico del control de acceso y de la configuración general del sistema.	Crear, modificar, bloquear y reactivar usuarios; asignar roles; consultar la bitácora de auditoría; administrar catálogos y parámetros generales.
Presidente	Representante principal de la asociación y responsable de dirigir las decisiones administrativas.	Consultar el padrón de miembros; registrar y actualizar proyectos; convocar reuniones y votaciones; asignar y revocar cargos directivos; consultar resultados.
Secretario	Responsable de mantener actualizados los registros administrativos y documentales.	Registrar y actualizar miembros; administrar períodos de directiva y cargos; programar reuniones; registrar asistencias; emitir listados oficiales.
Tesorero	Responsable del control de las aportaciones comunitarias y del seguimiento de los pagos.	Registrar aportaciones; corregir y anular registros con autorización; emitir comprobantes; consultar la recaudación por período.
Síndico	Responsable de supervisar el cumplimiento de los acuerdos y la transparencia de los procesos.	Consultar miembros, proyectos, aportaciones y votaciones; revisar resultados; acceder a información de auditoría sin alterar transacciones.
Miembro	Residente o integrante activo de la asociación comunal.	Consultar sus propios datos; consultar proyectos y reuniones; emitir un voto en los procesos habilitados; visualizar los resultados publicados.

Tabla 4. Roles de acceso del sistema y funciones autorizadas.

Durante este avance se consolidó una distinción conceptual que el primer avance no diferenciaba con claridad: los roles de acceso no son lo mismo que los cargos de la junta directiva. El rol determina qué puede hacer una cuenta dentro del sistema; el cargo directivo es un hecho institucional con vigencia, titular y período, que el sistema registra en las tablas cargo, periodo_directiva y miembro_cargo. Un mismo miembro puede ocupar el cargo de Tesorero durante un período y conservar un rol de acceso distinto, y el sistema modela ambas realidades por separado.

6. Requerimientos del sistema e historias de usuario

Las quince historias de usuario definidas en el primer avance se conservan sin alteración, por ser el enunciado original de la necesidad. La columna de estado que las acompaña indica el grado de implementación alcanzado tras este segundo avance y se desarrolla en el apartado 7.

Prioridad	Código	Historia de usuario	Criterios de aceptación	Estado
Alta	HU-001	Como secretario, quiero registrar un nuevo miembro para mantener actualizado el censo comunal.	El DUI no puede repetirse; nombres y apellidos son obligatorios; se registra fecha de ingreso y estado; el miembro puede consultarse después de guardar.	Implementada
Alta	HU-002	Como secretario, quiero actualizar los datos de un miembro para corregir cambios de contacto o residencia.	Solo se modifica un miembro existente; se validan correo y teléfono; se conserva el identificador; se registra fecha de actualización.	Implementada
Alta	HU-003	Como administrador, quiero crear usuarios y asignarles roles para controlar el acceso al sistema.	El nombre de usuario es único; la contraseña se almacena cifrada; cada usuario posee un rol activo; un usuario bloqueado no puede iniciar sesión.	Implementada
Alta	HU-004	Como tesorero, quiero registrar una aportación mensual para llevar el control de pagos de cada miembro.	Se selecciona un miembro; se registra mes, monto, fecha y método de pago; no se permite duplicar el mismo período; se genera comprobante.	Implementada
Alta	HU-005	Como tesorero, quiero consultar miembros con aportaciones pendientes para dar seguimiento a la morosidad.	El reporte filtra por período; muestra miembro, meses pendientes y total estimado; permite ordenar y exportar o imprimir.	Pendiente
Alta	HU-006	Como presidente, quiero registrar un proyecto comunitario para someterlo a evaluación y votación.	El proyecto tiene nombre, descripción, presupuesto y estado; queda asociado al usuario creador; puede pasar de borrador a propuesto.	Implementada
Alta	HU-007	Como presidente, quiero abrir una votación para que los miembros decidan sobre un proyecto.	Debe existir un proyecto activo; se define fecha de inicio y cierre; se crean al menos dos opciones; solo se habilita dentro del período configurado.	Implementada
Alta	HU-008	Como miembro, quiero emitir mi voto para participar en una decisión comunitaria.	Solo votan miembros activos; la votación debe estar abierta; se registra una sola respuesta por miembro y votación; el voto queda fechado.	Implementada
Alta	HU-009	Como síndico, quiero consultar los resultados de una votación para verificar la transparencia del proceso.	Se muestran votos por opción, total de participantes y porcentaje; no se revelan credenciales; el resultado se publica al cerrar la votación.	Implementada
Media	HU-010	Como secretario, quiero programar reuniones y registrar asistencia para documentar la participación de los miembros.	Se registra título, fecha, hora, lugar y tipo; la asistencia se asocia a miembros; se evita duplicar asistencia; se genera listado.	Implementada
Media	HU-011	Como secretario, quiero asignar cargos de directiva por período para conservar	El cargo, miembro y período deben existir; no se permite duplicar el mismo cargo	Implementada

Prioridad	Código	Historia de usuario	Criterios de aceptación	Estado
		el historial institucional.	activo en el mismo período; se conserva el historial.	da
Media	HU-012	Como miembro, quiero consultar mi estado de cuenta para conocer mis aportaciones registradas y pendientes.	Solo se muestran datos del miembro autenticado; se listan períodos, montos y estado; el saldo se calcula con reglas configuradas.	Pendiente
Media	HU-013	Como presidente, quiero consultar un tablero resumido para conocer el estado general de la asociación.	Muestra miembros activos, recaudación del período, proyectos por estado, votaciones activas y próximas reuniones.	Parcial
Media	HU-014	Como administrador, quiero consultar una bitácora de acciones para investigar cambios importantes.	Registra usuario, acción, entidad, fecha y detalle; no puede modificarse desde la interfaz; admite filtros por fecha y usuario.	Implementada
Baja	HU-015	Como usuario autorizado, quiero generar reportes para respaldar decisiones y presentar información a la comunidad.	Los reportes respetan permisos; permiten filtrar; incluyen fecha de emisión; pueden imprimirse o exportarse.	Pendiente

Tabla 5. Historias de usuario del proyecto y su estado tras el segundo avance.

6.1 Requerimientos funcionales consolidados

Código	Requerimiento funcional
RF-01	Autenticar usuarios y aplicar permisos según el rol asignado.
RF-02	Registrar, consultar, actualizar y desactivar miembros sin eliminar su historial.
RF-03	Administrar cargos y períodos de directiva.
RF-04	Registrar aportaciones mensuales y consultar estados de cuenta.
RF-05	Registrar proyectos comunitarios y controlar sus estados.
RF-06	Crear votaciones, opciones y períodos de participación.
RF-07	Validar un único voto por miembro en cada votación.
RF-08	Gestionar reuniones y asistencia.
RF-09	Generar reportes administrativos y de transparencia.
RF-10	Registrar acciones críticas en una bitácora de auditoría.

6.2 Requerimientos no funcionales

Código	Requerimiento no funcional
RNF-01	La aplicación será desarrollada en Java y utilizará MySQL como motor de base de datos.
RNF-02	Las contraseñas no se almacenarán en texto plano; se guardará únicamente un hash seguro.
RNF-03	Las operaciones críticas deberán ejecutarse mediante transacciones para mantener consistencia.
RNF-04	La interfaz deberá mostrar mensajes claros de validación y confirmación.
RNF-05	Las llaves foráneas y restricciones de unicidad deberán proteger la integridad referencial.
RNF-06	La aplicación debe responder a consultas normales en un tiempo razonable para un entorno académico local.
RNF-07	La solución debe organizarse por capas para separar interfaz, negocio y acceso a datos.

7. Alcance, limitaciones y límites

7.1 Funcionalidad implementada y verificada

Los módulos siguientes se encuentran implementados de extremo a extremo, desde la interfaz de escritorio hasta la persistencia, y cuentan con pruebas automatizadas sobre sus reglas de negocio:

- Autenticación de usuarios con contraseña cifrada, control de sesión y permisos por rol.
- Gestión del censo de miembros, con unicidad de documento y desactivación que preserva el historial.
- Administración de usuarios y roles, con protección de los perfiles base del sistema.
- Catálogo de cargos y administración de períodos de directiva, con una única directiva vigente a la vez.
- Asignación y revocación de cargos directivos por período, conservando el historial institucional.
- Registro de aportaciones mensuales con unicidad por miembro y período, corrección autorizada, anulación sin borrado físico y emisión de comprobante.
- Registro de proyectos comunitarios y control de sus estados.
- Votaciones, opciones de votación y emisión de votos con garantía de un solo voto por miembro.
- Programación de reuniones y registro de asistencia.
- Bitácora de auditoría que registra automáticamente toda operación de escritura.

7.2 Funcionalidad parcialmente implementada

El tablero resumido descrito en la historia HU-013 existe como pantalla y presenta los indicadores previstos, pero sus valores provienen todavía de un servicio de demostración y no de una consulta al sistema. Los datos que alimentarían ese tablero ya están disponibles en la interfaz de programación del servidor; falta conectarlos. Se declara como parcial y no como terminada.

La aplicación móvil está desarrollada en Flutter/Dart e integrada en el mismo repositorio mediante el módulo flutter-app. Actualmente implementa autenticación, pagos y aportaciones, consulta de reuniones y proyectos, votaciones —incluida la emisión de voto— y consulta de perfil y vivienda, consumiendo la interfaz REST del sistema. El módulo de notificaciones utiliza actualmente datos de muestra y queda pendiente su integración dinámica.

7.3 Funcionalidad pendiente para avances posteriores

Las consultas de cobranza descritas en las historias HU-005 y HU-012 no están implementadas. La razón es de fondo y conviene explicarla: el sistema registra y consulta lo que se ha pagado, pero una aportación pendiente no es un registro con otro estado, sino un registro que no existe. Determinarla exige contrastar el padrón de miembros activos contra los períodos esperados y restar lo efectivamente registrado, y esa consulta todavía no se ha construido. La misma situación aplica al módulo de reportes exportables de la historia HU-015.

7.4 Fuera del alcance del proyecto

- Pasarela bancaria o cobros en línea.
- Firma electrónica con validez legal.
- Integración con alcaldías, registros gubernamentales o servicios externos.
- Contabilidad completa de doble partida, nómina o inventario.
- Analítica empresarial avanzada o almacén de datos.

El primer avance situaba también fuera de alcance la existencia de una aplicación móvil. Ese límite se modificó durante el desarrollo: el repositorio incorpora una aplicación móvil desarrollada en Flutter, alojada en el módulo flutter-app del mismo repositorio, que consume la misma interfaz de programación del servidor. Se deja constancia del cambio por coherencia con el documento anterior.

8. Planificación

La planificación se organiza según las entregas del proyecto. Las actividades de los dos primeros avances corresponden a trabajo ya ejecutado, y su evidencia proviene del historial del repositorio y del tablero de Jira; las del tercer avance se presentan como planificadas.

8.1 Primer avance — análisis y diseño

Actividad	Responsable	Fecha	Evidencia
Análisis del problema y definición del objetivo general	Equipo	Julio 2026	Documento del primer avance
Definición de requerimientos e historias de usuario	Business Analyst y Product Owner	Julio 2026	Apartado 6 del primer avance
Diseño del modelo de datos relacional	Base de datos	Julio 2026	Archivo schema.sql
Estructura base del proyecto Java	Architect	Julio 2026	Commits iniciales del repositorio

8.2 Segundo avance — implementación y evidencia técnica

Actividad	Responsable	Fecha	Evidencia (ticket y commit)
Compilación reproducible con Maven Wrapper	Carlos Samayoa	12-sep-2026	SCRUM-319 · 68b8c88
Jerarquía de dominio Persona, Miembro y Residente	Carlos Samayoa	13-sep-2026	SCRUM-313 · 6640d6d
Clase abstracta de persistencia en archivos	Carlos Samayoa	13-sep-2026	SCRUM-39 · 475046a
Persistencia en archivos .dat	Carlos Samayoa	13-sep-2026	SCRUM-312 · 3a86806
Polimorfismo sobre el contrato de acceso a datos	Carlos Samayoa	13-sep-2026	SCRUM-314 · 0d349dd
Evidencia de encapsulamiento	Carlos Samayoa	13-sep-2026	SCRUM-317 · 7fbb114
Justificación de la concurrencia existente	Carlos Samayoa	13-sep-2026	SCRUM-315 · 4446d45

Actividad	Responsable	Fecha	Evidencia (ticket y commit)
Justificación de las colecciones empleadas	Carlos Samayoa	13-sep-2026	SCRUM-316 · a075efc
Documentación de la arquitectura por capas	Carlos Samayoa	13-sep-2026	SCRUM-318 · 448767d
Reconciliación del backlog histórico contra el código	Carlos Samayoa	13 y 14-sep-2026	Épicas SCRUM-179, 187, 197, 303 y 301
Revisión técnica del avance	Josué Castillo Nieves	14-sep-2026	Aprobación en los diez tickets
Recopilación de la evidencia del avance desde Git, Jira y el primer avance	Carlos Samayoa	14-sep-2026	Fuentes citadas en este documento
Construcción del documento de entrega del avance	Carlos Samayoa	13 y 14-sep-2026	SCRUM-321 · 02f00af y 76e416e
Coordinación de la integración con el revisor técnico	Carlos Samayoa	14-sep-2026	Pull request 28
Integración a la rama principal de desarrollo	Equipo	14-sep-2026	Pull request 28 · 936c7e2

8.3 Ciclo posterior a la integración y tercer avance — planificado

El ambiente de pruebas del proyecto se alimenta desde la rama principal de desarrollo. Por esa razón la validación funcional se ejecuta necesariamente después de integrar el avance y no antes: la aplicación que se somete a prueba es la que se despliega desde esa rama. Las actividades siguientes se encuentran asignadas y planificadas; ninguna de ellas se ha ejecutado todavía.

Actividad	Responsable	Fecha	Estado
Despliegue del avance integrado al ambiente de pruebas	Jonathan Alexis Amaya Hernández	Por definir	Planificado
Revisión funcional completa del build integrado	Jonathan Alexis Amaya Hernández	Por definir	Planificado
Registro en el tablero de los defectos encontrados	Jonathan Alexis Amaya Hernández	Por definir	Planificado
Corrección de los defectos reportados	Carlos Josué Samayoa Contreras	Por definir	Planificado
Despliegue de las correcciones al ambiente de pruebas	Jonathan Alexis Amaya Hernández	Por definir	Planificado
Revalidación de las correcciones desplegadas	Jonathan Alexis Amaya Hernández	Por definir	Planificado
Consultas de morosidad y estado de cuenta (HU-005 y HU-012)	Carlos Josué Samayoa Contreras	Por definir	Planificado
Conexión del tablero resumido a datos reales (HU-013)	Carlos Josué Samayoa Contreras	Por definir	Planificado
Módulo de reportes exportables (HU-015)	Carlos Josué Samayoa Contreras	Por definir	Planificado
Validación funcional de las historias incorporadas	Jonathan Alexis Amaya Hernández	Por definir	Planificado
Conclusión de la reconciliación del backlog histórico	Carlos Josué Samayoa Contreras	Por definir	Planificado
Atención de los hallazgos técnicos registrados durante el avance	Carlos Josué Samayoa Contreras	Por definir	Planificado

Actividad	Responsable	Fecha	Estado
Coordinación de los cambios de código con el revisor técnico	Carlos Josué Samayoa Contreras	Por definir	Planificado
Documentación y presentación final	Equipo	Por definir	Planificado

Tabla 6. Distribución de actividades del equipo según las entregas del proyecto.

La secuencia acordada para el ciclo de calidad es la siguiente: revisión técnica del código, integración a la rama de desarrollo, despliegue al ambiente de pruebas, validación funcional, registro de los defectos en el tablero, corrección desde la rama de desarrollo actualizada y nueva validación de lo corregido. Las correcciones no se realizan sobre la rama de este avance, que ya cumplió su propósito.

Las fechas se consignan como «por definir» porque el calendario académico correspondiente no ha sido comunicado. La única fecha oficial conocida es la de entrega de este segundo avance: 27 de septiembre de 2026 a las 13:00.

9. Cronograma — diagrama de Gantt

El cronograma se representa según las entregas del proyecto. Cada fila corresponde a una línea de trabajo y cada columna a una quincena. Las barras de color indican el período en que la actividad se ejecutó o está prevista.

Actividad	Julio		Agosto		Septiembre		Octubre		Noviembre	
2026	1ª	2ª	1ª	2ª	1ª	2ª	1ª	2ª	1ª	2ª
AVANCE 1 · Análisis y diseño										
Análisis del problema y objetivos										
Requerimientos e historias de usuario										
Modelo de datos y estructura base										
AVANCE 2 · Implementación y evidencia técnica										
Construcción de módulos del sistema										
Aplicación móvil en Flutter (módulo flutter-app)										
Evolución del diseño de clases y herencia										
Clases abstractas y persistencia .dat										
Polimorfismo sobre el contrato DAO										
Documentación técnica del avance										
Revisión técnica e integración										
Entrega del segundo avance (27-sep-2026)										
QA Y DESPLIEGUE · posterior a la integración										
Despliegue del avance al ambiente de pruebas (JA)										
Revisión funcional del build integrado (JA)										
Corrección de defectos reportados (CS)										
Despliegue de correcciones (JA)										
Revalidación de correcciones (JA)										
AVANCE 3 · Funcionalidad pendiente										
Consultas de cobranza HU-005 y HU-012 (CS)										
Tablero con datos reales HU-013 (CS)										
Módulo de reportes HU-015 (CS)										
Validación funcional de lo incorporado (JA)										
ENTREGA FINAL										
Endurecimiento técnico y hallazgos										



Figura 1. Diagrama de Gantt del proyecto según las entregas de avances.



La construcción principal correspondiente al Segundo Avance se consolidó hasta el 14 de septiembre de 2026. Posteriormente, durante la segunda quincena de septiembre, se realizaron actividades de integración, estabilización y validación funcional previas a la entrega del 27 de septiembre. Las barras del tercer avance y de la entrega final se presentan como planificadas: su ubicación es tentativa dado que el calendario académico correspondiente aún no ha sido comunicado.

Las iniciales indican el responsable asignado. CS corresponde a Carlos Josué Samayoa Contreras, a cargo de la arquitectura, el desarrollo, la evolución del código para cumplir los criterios de la rúbrica, la gestión del backlog y la corrección de defectos. JA corresponde a Jonathan Alexis Amaya Hernández, a cargo de la validación funcional y del despliegue de los avances y sus correcciones. El bloque de calidad aparece después de la integración porque el ambiente de pruebas se alimenta desde la rama principal de desarrollo.

10. Entradas y salidas del sistema

Se presentan las entradas y salidas de los módulos principales, asociadas a las historias de usuario que les dan origen. No se detallan las quince historias con la misma extensión: se agrupan por módulo para conservar la legibilidad del documento.

Módulo (historias)	Entradas principales	Salidas esperadas
Censo de miembros (HU-001, HU-002)	Tipo y número de documento, nombres, apellidos, teléfono, correo, dirección, vivienda y fecha de ingreso.	Ficha del miembro registrada; confirmación de la operación; miembro disponible en las búsquedas del padrón; mensaje de rechazo si el documento ya existe.
Usuarios y roles (HU-003)	Nombre de usuario, contraseña inicial, rol asignado, miembro vinculado y estado.	Cuenta creada con la contraseña almacenada como hash; permisos aplicados según el rol; mensaje de validación o de duplicidad.
Aportaciones (HU-004)	Miembro, período mensual en formato año-mes, monto, fecha de pago, método de pago y referencia.	Aportación registrada; comprobante oficial con número correlativo; total recaudado actualizado; rechazo si el período ya fue registrado para ese miembro.
Cobranza (HU-005, HU-012)	Período o rango de períodos y filtros opcionales por miembro.	Listado de miembros con períodos pendientes y total estimado; estado de cuenta individual. Funcionalidad pendiente de implementación.
Proyectos comunitarios (HU-006)	Nombre, descripción, presupuesto, responsable, fechas y estado.	Proyecto registrado con su identificador; ficha del proyecto; control de su estado.
Votaciones (HU-007, HU-008, HU-009)	Proyecto asociado, título, fechas de apertura y cierre, opciones de voto; para el votante, la opción seleccionada.	Votación creada y habilitada en su período; confirmación del voto con fecha y hora; prevención de voto duplicado; conteo por opción, porcentajes y participación total.
Reuniones y asistencia (HU-010)	Título, fecha, hora, lugar, tipo de reunión y lista de miembros convocados.	Reunión programada; listado de asistencia; reporte de participación.
Cargos de directiva (HU-011)	Miembro, cargo, período directivo y fechas de vigencia.	Asignación registrada; historial de la directiva por período; rechazo si el cargo ya está ocupado en ese período.
Tablero resumido (HU-013)	Sesión del usuario autenticado.	Indicadores generales de la asociación. Actualmente presenta valores de demostración.
Bitácora de auditoría (HU-014)	Operación de escritura ejecutada por un usuario autenticado.	Asiento con usuario, acción, entidad, identificador del registro, fecha y detalle; consulta con filtros.

Tabla 7. Entradas y salidas por módulo del sistema.

11. Declaración de entidades del sistema

El modelo de dominio está compuesto por diecisiete clases de entidad, en correspondencia con las diecisiete tablas del esquema relacional. Respecto del primer avance se incorporaron la clase abstracta Persona y la entidad Residente, producto de la evolución del diseño descrita en el apartado 12, además de las entidades Vivienda y Rol que el modelo ya contemplaba.

Entidad	Descripción	Atributos principales
Persona	Clase abstracta que agrupa a toda persona vinculada a una vivienda de la comunidad.	idVivienda; contrato abstracto de nombre completo y condición de asociado
Miembro	Asociado formal de la asociación comunal. Extiende Persona.	idMiembro, dui, tipoDocumento, paisOrigen, nombres, apellidos, teléfono, correo, dirección, fechaIngreso, estado
Residente	Persona que habita una vivienda sin ser necesariamente asociada. Extiende Persona.	idResidente, idMiembro (opcional), nombreCompleto, tipoPersona
Vivienda	Unidad habitacional registrada en la comunidad.	idVivienda, código, dirección, estado
Usuario	Credencial de acceso al sistema, vinculada a un miembro.	idUsuario, nombreUsuario, claveHash, idRol, idMiembro, estado, ultimoAcceso
Rol	Perfil de autorización que determina los permisos de una cuenta.	idRol, nombre, descripción
Cargo	Puesto de la junta directiva susceptible de ser ocupado.	idCargo, nombre, descripción, nivelJerarquico, activo
PeriodoDirectiva	Período de gobierno de una junta directiva.	idPeriodo, nombre, fechaInicio, fechaFin, estado
MiembroCargo	Asignación de un cargo a un miembro dentro de un período.	idMiembroCargo, idMiembro, idCargo, idPeriodo, fechaAsignacion, fechaFin, motivoSalida, estado
Aportacion	Pago mensual registrado a nombre de un miembro.	idAportacion, idMiembro, idProyecto (opcional), periodoMes, monto, fechaPago, metodoPago, referencia, estado
Proyecto	Iniciativa comunitaria sujeta a evaluación y votación.	idProyecto, nombre, descripción, presupuesto, fechas, estado, idUsuarioCreador
Votacion	Proceso de decisión abierto a los miembros.	idVotacion, idProyecto, título, descripción, fechaInicio, fechaFin, estado
OpcionVotacion	Alternativa sobre la que se emite el voto.	idOpcion, idVotacion, texto, orden
Voto	Respuesta individual de un miembro en una votación.	idVoto, idVotacion, idOpcion, idMiembro, fechaVoto
Reunion	Asamblea o sesión convocada por la asociación.	idReunion, título, descripción, fecha, hora, lugar, tipo, estado
Asistencia	Registro de la presencia de un miembro en una reunión.	idAsistencia, idReunion, idMiembro, estado, observación
Bitacora	Asiento de auditoría de las operaciones de escritura del sistema.	idBitacora, idUsuario, acción, entidad, idRegistro, detalle, fecha

Tabla 8. Entidades del dominio tras el segundo avance.

12. El proyecto: evolución técnica del segundo avance

12.1 Tecnología empleada

El proyecto se desarrolla sobre la plataforma Java y se organiza como un proyecto Maven de varios módulos. La tabla siguiente resume el conjunto tecnológico efectivamente utilizado.

Componente	Tecnología	Función en el sistema
Lenguaje y plataforma	Java 21	Base del sistema; permite aplicar los conceptos de programación orientada a objetos que el avance evalúa.
Servidor de aplicación	Javalin sobre Jetty	Expone la interfaz de programación del sistema mediante noventa y cinco rutas REST.
Interfaz de escritorio	JavaFX	Cliente principal de la asociación; treinta y tres vistas con su respectivo controlador.
Base de datos	MySQL con InnoDB	Almacenamiento operativo del sistema.
Acceso a datos	JDBC	Comunicación entre la aplicación y la base de datos.
Persistencia alternativa	Archivos .dat por serialización	Origen de datos autónomo incorporado en este avance.
Comunicación	REST con intercambio en JSON	Enlace entre los clientes y el servidor.
Construcción	Maven con Maven Wrapper	Gestión de dependencias y compilación reproducible.
Aplicación móvil	Flutter / Dart	Cliente móvil para residentes, integrado en el mismo repositorio en el módulo flutter-app; consume la misma interfaz REST del servidor.

Tabla 9. Conjunto tecnológico del proyecto.

12.2 Aclaraciones técnicas validadas por el facilitador

Antes de iniciar la implementación se plantearon tres consultas al facilitador, Ing. Daniel Enrique Guevara Gómez, cuyas respuestas orientaron las decisiones descritas en este apartado.

Consulta	Respuesta del facilitador	Efecto en la implementación
El sistema ya funciona con MySQL. ¿La persistencia en archivos .dat debe reemplazar la base de datos, pueden coexistir ambas, o basta con implementar un proceso real de archivos?	«Funcionando los dos a la vez estaría bien.»	MySQL se conserva como origen operativo. La persistencia en archivos se incorpora como una implementación adicional del mismo contrato de acceso a datos, en coexistencia y sin sustituir a la base.
¿Es válido el conjunto Java 21, Javalin, JavaFX, MySQL y REST, aunque no corresponda literalmente a una fila de la tabla de tecnologías recomendadas?	«No es obligación, son recomendaciones; si ya lo tienen así no hay problema.»	El conjunto tecnológico se documenta como la elección del proyecto y no como una desviación respecto de la guía.
¿Sirve justificar la concurrencia ya existente en el sistema, o debe construirse una operación concurrente nueva?	«Sí, sirve justificar y documentar lo que ya tienen avanzado.»	No se añadió ningún mecanismo concurrente artificial. Se inventarió y justificó técnicamente la concurrencia que el sistema ya empleaba para resolver problemas reales.

Tabla 10. Consultas planteadas al facilitador y su incidencia en el avance.

12.3 Evolución del primer avance y diseño de clases

El primer avance entregó el diseño de las clases; este segundo avance lo llevó a una implementación que resuelve una carencia concreta del modelo anterior. El diseño original representaba únicamente al miembro asociado, pero el modelo de datos ya admitía que una vivienda albergara personas sin condición de asociado: la columna que vincula al residente con su ficha de miembro es opcional. Esa realidad no tenía representación en el código.

Aspecto del diseño	Primer avance	Segundo avance
Representación de las personas	Una sola clase, Miembro	Jerarquía Persona, Miembro y Residente
Herencia propia del dominio	No existía	Clase abstracta Persona con dos subclases
Clases abstractas	No existían	Persona y AbstractArchivoDAO
Origen de datos	Únicamente MySQL	MySQL y archivos .dat sobre el mismo contrato
Uso del contrato de acceso a datos	Declarado, no ejercido	Consumido de forma polimórfica por la capa de servicio
Pruebas automatizadas	Sin línea base establecida	Ciento ochenta y nueve pruebas ejecutadas

Tabla 11. Evolución del diseño entre el primer y el segundo avance.

12.4 Encapsulamiento

Las diecisiete clases de entidad del dominio declaran ciento seis atributos privados y ninguno público. Ningún dato del dominio es accesible directamente desde fuera de su clase. Sin embargo, el encapsulamiento del sistema va más allá de esa convención: se emplea para impedir que un objeto alcance un estado inválido.

El estado de un miembro no se representa con una cadena de texto sino con un tipo enumerado. La diferencia es sustantiva: si fuera texto, valores como «Activo», «ACITVO» o una cadena vacía compilarían sin error y el problema aparecería en ejecución. Con el tipo enumerado, el compilador lo impide.

```
public class Miembro extends Persona implements Serializable {
    public enum Estado { ACTIVO, INACTIVO }

    private Integer idMiembro;
    private String dui;
    private Estado estado;

    public boolean estaActivo() { return estado == Estado.ACTIVO; }
}
```

El nombre completo del miembro no se almacena como atributo: se calcula a partir de los nombres y apellidos. De este modo no puede quedar desincronizado respecto de los datos que lo componen.

Un segundo caso ilustra el encapsulamiento a escala de clase. La utilidad encargada del cifrado de contraseñas declara públicos únicamente dos de sus siete métodos; el algoritmo, el número de iteraciones y la generación de la sal quedan encerrados, y su constructor privado impide instanciarla. La entidad Usuario almacena el atributo claveHash y en ningún momento la contraseña en claro.

12.5 Herencia

La jerarquía implementada responde a una distinción real de la asociación comunal: no todas las personas que habitan una vivienda son asociadas. Un hijo, un inquilino o un familiar residen en la comunidad sin tener ficha de miembro, y el modelo de datos ya lo contemplaba mediante una referencia opcional.

```
public abstract class Persona implements Serializable {  
  
    private Integer idVivienda;           // privado incluso para las subclases  
  
    protected Persona() { }              // no se instancia desde fuera  
  
    public abstract String getNombreCompleto();  
    public abstract boolean esAsociado();  
}
```

De Persona derivan dos subclases. Miembro representa al asociado formal y responde siempre de forma afirmativa a la pregunta de si es asociado. Residente representa a quien habita una vivienda y responde según tenga o no ficha de miembro vinculada.

Tres decisiones merecen explicación. En primer lugar, el atributo de vivienda es privado incluso para las subclases, que deben acceder a él mediante sus métodos; en segundo lugar, los constructores son protegidos, porque no existe «una persona» suelta en el padrón, sino siempre un miembro o un residente; en tercer lugar, la clase base solo exige a sus subclases responder cómo se llaman y si son asociadas, sin imponerles cómo almacenarlo.

Conviene señalar también lo que deliberadamente no se incorporó a la clase base. El documento de identidad, el teléfono, el correo y la fecha de ingreso permanecen únicamente en Miembro, porque el registro de residentes no tiene esas columnas. Elevarlas a Persona para robustecer la jerarquía habría significado inventar información que el dominio no posee.

Por la misma razón, la credencial de acceso no hereda de Persona. Un usuario no tiene nombres ni vivienda: es una credencial que referencia a un miembro. La relación correcta es de composición y no de herencia.

12.6 Polimorfismo

El sistema define un contrato genérico de acceso a datos. En el primer avance ese contrato existía y era implementado por los objetos de acceso a datos, pero nunca se usaba como tipo: los servicios declaraban siempre la implementación concreta. El polimorfismo estaba declarado y no ejercido.

Este avance introduce un servicio de consulta del padrón que depende exclusivamente de la abstracción:

```
public class CensoMiembrosService {  
  
    private final DAO<Miembro, Integer> origen;    // el contrato, no la implementacion  
  
    public CensoMiembrosService(DAO<Miembro, Integer> origen) {  
        this.origen = Objects.requireNonNull(origen);  
    }  
  
    public List<MiembroResponse> findAll() {  
        return origen.findAll().stream().map(MiembroResponse::from).toList();  
    }  
}
```

El servicio no sabe de dónde provienen los datos. La implementación concreta se inyecta desde fuera por el constructor, y puede ser tanto el objeto de acceso basado en JDBC contra MySQL como el basado en archivos. No existe ninguna conversión de tipo ni comprobación de instancia en la clase.

La evidencia del despacho polimórfico es una prueba parametrizada que ejecuta la misma lógica contra las dos implementaciones y verifica que el resultado observable es idéntico, incluido el caso del miembro inexistente. Dos pruebas adicionales comprueban por reflexión que el servicio declara un único colaborador, de tipo contrato, y que ninguna de sus firmas menciona una implementación concreta.

Corresponde precisar el alcance real de esta capacidad. En ejecución, el servicio recibe siempre la implementación basada en MySQL. La rama de archivos está construida, probada y disponible, pero no se activa de forma automática porque no existe sincronización entre ambos orígenes; activarla podría presentar información desactualizada. Lo demostrable, y lo que el criterio evalúa, es que la capa de servicio no depende de cuál de las dos esté conectada.

12.7 Programación concurrente

Conforme a la aclaración del facilitador, no se construyó ninguna operación concurrente artificial. El sistema ya empleaba concurrencia para resolver problemas concretos de la asociación, y este avance la inventarió y la justificó técnicamente.

Mecanismo	Ocurrencias	Función en el sistema
Tareas en segundo plano (Task)	58	Toda llamada al servidor se ejecuta fuera del hilo de interfaz.
Hilos explícitos	55	Ejecución de las tareas; 33 de ellos declarados como demonio.
Retorno seguro al hilo de interfaz	9	Actualización de la pantalla desde un hilo secundario.
Planificador de tarea periódica	1	Verificación desatendida de actualizaciones del cliente.
Variables atómicas y volátiles	3	Estado compartido sin bloqueo.
Hilos virtuales de Java 21	3	Descarga de paquetes de actualización.
Bloques sincronizados	3	Emisión de votos y creación perezosa de la conexión.

Tabla 12. Inventario de mecanismos de concurrencia presentes en el sistema.

El caso más representativo: el pase de lista en asamblea

Durante una asamblea, el secretario registra la asistencia de decenas de miembros frente a la gente. Si la interfaz se congelara en cada consulta al servidor, el trabajo sería inviable. La carga del listado se ejecuta en una tarea en segundo plano sobre un hilo con nombre propio, de modo que la ventana permanece utilizable mientras los datos viajan. Es, además, el caso demostrable en vivo: basta mover la ventana o desplazar la tabla mientras la información se carga.

El caso de diseño propio: el verificador de actualizaciones

El servicio de actualización del cliente utiliza un planificador de un solo hilo declarado como demonio, de forma que la verificación periódica no impide que la aplicación cierre. Una variable atómica garantiza que no se solapen dos comprobaciones, y su liberación está situada en un bloque final para que una falla no deje el mecanismo bloqueado. Una variable volátil asegura la visibilidad de la última versión ofrecida entre hilos.

Este ejemplo se presenta en primer lugar durante la defensa por una razón deliberada: un evaluador puede objetar, con razón, que las tareas en segundo plano son el patrón que la biblioteca de interfaz obliga a usar. El planificador, en cambio, es diseño propio del equipo.

Concurrencia con límite declarado: la emisión del voto

La emisión de votos utiliza un bloque sincronizado para impedir que un mismo miembro vote dos veces. Es importante declarar su límite con precisión: la sincronización protege únicamente dentro de un mismo proceso. La garantía efectiva la aporta una restricción de unicidad en la base de datos sobre la pareja votación y miembro. Presentar el bloque sincronizado como garantía completa sería técnicamente incorrecto.

Concurrencia y paralelismo

El sistema emplea concurrencia y no paralelismo, y la distinción es pertinente. Lo que se traslada fuera del hilo de interfaz es espera de entrada y salida, no cálculo. En consecuencia, no se afirma mejora de velocidad por uso de múltiples núcleos: el proyecto no utiliza flujos paralelos ni reservas de hilos para cómputo, y su planificador es explícitamente de un solo hilo.

12.8 Clases abstractas

El sistema incorpora dos clases abstractas de naturaleza distinta. La primera, *Persona*, pertenece al dominio y fue descrita en el apartado de herencia. La segunda pertenece a la capa de persistencia y es genérica:

```
public abstract class AbstractArchivoDAO<T extends Serializable, ID>
    implements DAO<T, ID> {

    private final Path archivo;                // la ruta no cambia

    protected abstract ID    idDe(T entidad);
    protected abstract void asignarIdentidad(T entidad, List<T> existentes);

    protected final List<T> leerTodo();        // reutilizable, no redefinible
    protected final void    escribirTodo(List<T> registros);
}
```

La clase concentra el comportamiento común de la persistencia en archivos —apertura, serialización, recuperación y manejo de errores de entrada y salida— e implementa el contrato completo de acceso a datos. Una subclase concreta solo debe aportar dos métodos: cómo obtener el identificador de una entidad y cómo asignar uno nuevo.

Los tres niveles de visibilidad fueron elegidos, no heredados de una convención. La ruta del archivo es privada, de modo que las subclases la reciben por constructor y no pueden alterarla. La lectura y la escritura son accesibles a las subclases pero no redefinibles, lo que impide que una implementación concreta rompa el formato del archivo para las demás. Los dos métodos abstractos son los únicos puntos de extensión.

La prueba de esta clase no emplea una entidad del dominio sino una entidad auxiliar creada para el efecto, sin relación alguna con la asociación comunal. Esa decisión es la que demuestra que la abstracción es genuinamente reutilizable: si solo funcionara con *Miembro*, no sería una clase abstracta reutilizable sino un objeto de acceso a datos con pasos adicionales.

12.9 Colecciones

El sistema emplea las estructuras de datos de la plataforma de manera extendida. El inventario siguiente corresponde al código de producción de ambos módulos.

Estructura	Ocurrencias	Uso principal en el sistema
List	281	Padrón de miembros y todo listado que conserva orden.
ArrayList	61	Construcción de listados en la capa de acceso a datos.
Map	31	Acceso por clave; control de intentos fallidos de inicio de sesión.
Optional	46	Retorno de las búsquedas por identificador, en lugar de valores nulos.
Flujos (stream)	78	Transformación de entidades en objetos de transferencia.
Set	3	Conjunto de roles base del sistema, inmutable y sin duplicados.
ObservableList	16	Enlace entre los datos y las tablas de la interfaz.
FilteredList	18	Vistas filtradas sin copiar la colección de origen.

Tabla 13. Estructuras de datos empleadas y su función.

Conviene distinguir cuatro conceptos que suelen confundirse. Las colecciones almacenan elementos; Optional no es una colección sino un contenedor de cero o un valor, empleado para que la ausencia de un registro sea explícita en lugar de devolver un valor nulo; los flujos tampoco almacenan, sino que describen una transformación que se consume una sola vez; y los arreglos, de tamaño fijo, aparecen únicamente en la frontera con la biblioteca de serialización y se convierten a lista en la misma línea.

El caso más ilustrativo es el de la papeleta de votación. Al reordenar las opciones, el cliente no envía ningún campo de posición al servidor: envía una lista de identificadores, y el orden de esa lista es el orden nuevo. La estructura transporta la información sin necesidad de un atributo adicional.

Un segundo caso es el conjunto de roles base. Se emplea un conjunto inmutable porque las tres propiedades que se necesitan son exactamente las que ofrece: unicidad, comprobación de pertenencia e imposibilidad de modificación en ejecución.

12.10 Arquitectura por capas

El proyecto separa las responsabilidades de presentación, control, lógica de negocio y acceso a datos, y emplea objetos de entidad, de transferencia y de modelo según corresponde a cada capa.

Cliente JavaFX	Servidor	Persistencia
-----	-----	-----
Vista FXML	→ JwtAuthMiddleware	
Controlador JavaFX	→ Controlador HTTP	→ DAO → MySQL
Cliente de API	→ Servicio (reglas)	→ DAO → archivo .dat
(REST / JSON)	Interceptor de bitacora	

Figura 2. Organización por capas del sistema.

Capa	Ubicación	Elementos
Presentación	Módulo de escritorio	33 vistas FXML y 35 controladores de interfaz
Acceso remoto	Módulo de escritorio	14 clientes de API que aíslan la comunicación HTTP
Control	Módulo de servidor	19 controladores HTTP sobre 95 rutas REST
Negocio	Módulo de servidor	19 servicios con las reglas del dominio
Acceso a datos	Módulo de servidor	17 objetos de acceso a datos y 2 de persistencia en archivo
Modelo	Módulo de servidor	17 entidades de dominio y 44 objetos de transferencia

Tabla 14. Distribución de responsabilidades por capa.

Las reglas de dependencia entre capas no se declaran: se verificaron sobre el código. El módulo de escritorio no contiene ninguna referencia a la base de datos; los objetos de acceso a datos no importan servicios; y las entidades no dependen de objetos de transferencia ni de la capa de persistencia.

El reparto de responsabilidades puede seguirse en una operación concreta. Al dar de baja a un miembro, el controlador HTTP recibe la petición y no conoce los estados posibles; el servicio traduce el texto recibido al tipo enumerado correspondiente y rechaza cualquier valor que no sea válido; y el objeto de acceso a datos ejecuta, dentro de una transacción, la actualización del miembro y la de su cuenta de usuario. Sin esa transacción podría quedar un miembro inactivo cuya cuenta siguiera habilitada para entrar al sistema.

El registro de auditoría se resuelve como una capa transversal. Un interceptor posterior a la ejecución de cada ruta anota en la bitácora toda operación de escritura que haya concluido correctamente, tomando el usuario del contexto autenticado. Ningún controlador ni servicio contiene una sola línea de auditoría: escribirla en cada método de escritura supondría cerca de cuarenta puntos que mantener sincronizados.

12.11 Persistencia mediante archivos .dat

Conforme a la aclaración del facilitador, la persistencia en archivos coexiste con la base de datos y no la reemplaza. Se incorporó como una implementación adicional del mismo contrato de acceso a datos que ya empleaba el sistema.

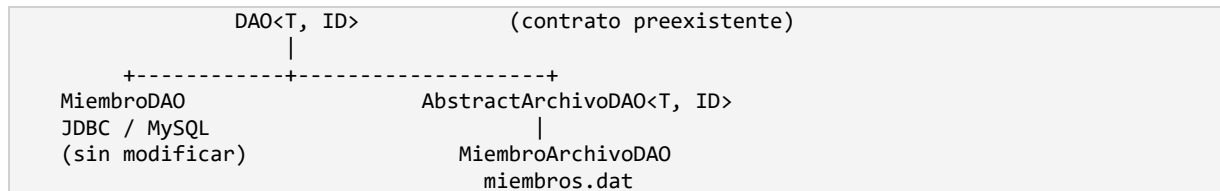


Figura 3. Coexistencia de los dos orígenes de datos sobre un mismo contrato.

La implementación basada en archivos almacena el censo mediante serialización de objetos. Todas las entidades de la jerarquía declaran su identificador de versión de serialización de forma explícita. La clase concreta declara únicamente los dos métodos que la clase abstracta exige; la totalidad de las operaciones de consulta, alta, modificación y baja se heredan.

La prueba principal verifica que el archivo funciona como origen de datos y no como simple respaldo: guarda información con una instancia, la descarta por completo, crea otra apuntando al mismo archivo, recupera el censo sin intervención de la base de datos y continúa operando con el correlativo leído del propio archivo.

Tres precisiones sostienen la honestidad de esta evidencia. La primera es que no existe sincronización entre la base de datos y los archivos, y el documento no la afirma. La segunda es que en ejecución el sistema utiliza la implementación basada en MySQL; la de archivos se ejercita mediante las pruebas. La tercera es una limitación conocida y documentada del generador de identificadores, que los asigna a partir del mayor existente: si se elimina el registro de identificador más alto, ese valor queda disponible nuevamente. Existe una prueba dedicada que deja constancia del comportamiento. Evitarlo exigiría un contador independiente dentro del archivo, una complejidad que el alcance no justifica.

12.12 Versionamiento con Git

El trabajo del avance se realizó sobre una rama independiente, con un commit por cada tarea del tablero y la clave de Jira en el mensaje, de modo que cada cambio es trazable hasta su ticket. La integración a la rama principal de desarrollo se realizó mediante una solicitud de incorporación revisada.

Commit	Contenido	Ticket
68b8c88	Maven Wrapper para compilación reproducible	SCRUM-319
6640d6d	Jerarquía de dominio Persona, Miembro y Residente	SCRUM-313
475046a	Clase abstracta de persistencia en archivos	SCRUM-39
3a86806	Persistencia de miembros en archivos .dat	SCRUM-312
0d349dd	Consulta del censo a través del contrato de acceso a datos	SCRUM-314
7fbb114	Evidencia de encapsulamiento	SCRUM-317
4446d45	Justificación de la concurrencia existente	SCRUM-315
a075efc	Justificación del uso de colecciones	SCRUM-316
448767d	Documentación de la arquitectura por capas	SCRUM-318
02f00af	Documento del segundo avance	SCRUM-321
a1e4433	Registro de la revisión técnica	SCRUM-321
76e416e	Participación, nombres y roles del equipo	SCRUM-321

Tabla 15. Commits del segundo avance y su trazabilidad con el tablero.

El documento correspondiente al avance queda versionado en la raíz del proyecto, conforme al requisito de la guía, junto con los cuatro documentos de evidencia técnica que lo respaldan y que residen en la carpeta de documentación.

12.13 Integración y funcionamiento del proyecto

El repositorio incorpora el Maven Wrapper, de modo que el proyecto compila y ejecuta sus pruebas sin necesidad de una instalación previa de Maven. La verificación se realizó además desde un clon limpio del repositorio, para comprobar que la reproducibilidad no depende del equipo de desarrollo.

Módulo	Pruebas ejecutadas	Correctas	Errores
Servidor (asociacion-api)	183	183	0
Cliente de escritorio	6	5	1
Total	189	188	1

Tabla 16. Resultado de la ejecución de la batería de pruebas.

La progresión de la línea base durante el avance fue de 144 pruebas iniciales a 189, con un incremento de cuarenta y cinco pruebas nuevas, sin que ninguna prueba existente resultara afectada.

El único error corresponde a una prueba de navegación de la interfaz que requiere un archivo de configuración con credenciales, deliberadamente excluido del control de versiones por contener un token de acceso. Se trata de una condición del entorno y no de un defecto lógico: el error ya se presentaba en la línea base inicial y está registrado para su resolución en la fase de endurecimiento técnico posterior.

12.14 Documentación técnica y preparación de la presentación

Además de este documento, el avance produjo cuatro documentos de evidencia técnica versionados junto con el código, que sustentan los apartados anteriores con referencias precisas a archivo y línea:

Documento	Criterio que sustenta	Ticket
encapsulamiento.md	Encapsulamiento	SCRUM-317
concurrency.md	Programación concurrente	SCRUM-315
coleccioness.md	Colecciones	SCRUM-316
arquitectura.md	Arquitectura por capas	SCRUM-318
AVANCE_2.md	Consolidación de los doce criterios	SCRUM-321

Tabla 17. Documentos de evidencia técnica versionados con el proyecto.

La guía advierte que durante la presentación se formularán preguntas a integrantes elegidos al azar para explicar el código. Con ese fin, cada uno de los cuatro documentos incluye una sección final que indica qué archivos abrir y en qué orden para demostrar el criterio correspondiente. El equipo reconoce que este material reduce el riesgo pero no lo elimina: su utilidad depende de que los integrantes lo estudien antes de la defensa.

13. Conclusiones

Las conclusiones siguientes se refieren específicamente al trabajo desarrollado durante este segundo avance.

- La evolución del diseño de clases no consistió en añadir jerarquías para satisfacer un criterio de evaluación, sino en representar una realidad que el modelo de datos ya admitía y el código ignoraba: que una vivienda puede albergar personas sin condición de asociado. La jerarquía Persona, Miembro y Residente surgió de esa carencia concreta.
- Resultó igualmente formativo decidir qué no debía heredar. Mantener el documento de identidad y los datos de contacto únicamente en Miembro, y modelar la credencial de acceso como una referencia y no como una subclase, evitó construir una jerarquía aparente que el dominio no respalda.
- El polimorfismo dejó de ser una declaración para convertirse en una capacidad verificable. Que la misma prueba se ejecute sin cambios contra la base de datos y contra el archivo demuestra que la capa de servicio depende del contrato y no de la tecnología de almacenamiento.
- La coexistencia de MySQL con la persistencia en archivos, conforme a la aclaración del facilitador, permitió incorporar el segundo origen de datos sin desmontar lo ya construido. El equipo optó por no activar la conmutación automática entre ambos, porque sin sincronización podría presentarse información desactualizada al usuario.
- Documentar la concurrencia existente exigió más rigor que haber creado un hilo de demostración. Obligó a explicar qué problema resuelve cada mecanismo, a distinguir concurrencia de paralelismo y a declarar el límite real del bloque sincronizado que protege la emisión del voto, cuya garantía efectiva reside en una restricción de la base de datos.
- La arquitectura por capas se verificó en lugar de afirmarse. Comprobar sobre el código que el cliente no accede a la base de datos y que las entidades no dependen de otras capas aportó más certeza que cualquier diagrama.
- La disciplina de versionamiento —un commit por tarea, con la clave del ticket en el mensaje— permitió que la planificación de este documento se reconstruyera a partir de evidencia real y no de estimaciones.
- El avance también dejó constancia de lo que no está terminado. Las consultas de morosidad y de estado de cuenta continúan pendientes, y el tablero resumido presenta todavía valores de demostración. Declararlo explícitamente resultó preferible a presentar como concluido aquello que el equipo sabe incompleto.
- Finalmente, la distribución del conocimiento sigue siendo el principal riesgo del proyecto. La mayor parte del código fue escrita por un solo integrante, y las secciones de defensa incorporadas a los documentos de evidencia buscan mitigarlo, aunque su eficacia depende de que el equipo las estudie antes de la presentación.

14. Bibliografía

- Oracle. (s. f.). Java Documentation. Documentación oficial del lenguaje y la plataforma Java.
- Oracle. (s. f.). MySQL 8.0 Reference Manual. Documentación oficial de MySQL.
- Oracle. (s. f.). Java Platform, Standard Edition: Concurrency. Documentación oficial sobre hilos, sincronización y ejecutores.
- OpenJFX. (s. f.). JavaFX API Documentation. Documentación oficial de la biblioteca de interfaz gráfica.
- Javalin. (s. f.). Javalin Documentation. Documentación oficial del marco de trabajo web empleado en el servidor.
- Schwaber, K., y Sutherland, J. (2020). The Scrum Guide.
- Pressman, R. S., y Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach. McGraw-Hill.
- Universidad Evangélica de El Salvador. Guía de contenido para el primer avance del proyecto de Programación.
- Universidad Evangélica de El Salvador. Guía de contenido para el segundo avance del proyecto de Programación II.

Las referencias a documentación oficial se consignan sin fecha de consulta ni dirección electrónica específica, por tratarse de documentación viva cuya versión publicada varía. Se citan por su título oficial para que puedan localizarse de forma inequívoca.